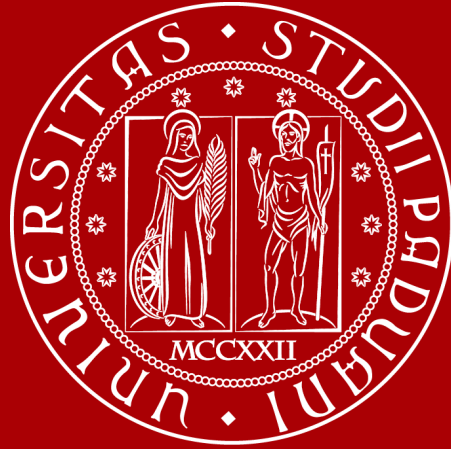




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

RL Cooperative Transportation using fixed-wings drones

End of July status



Original Plan

Formulation 1:

- Action Space: $\lambda \in \mathbb{R}^n$
 - Keep wrench controller and nullspace matrix. Run decentralized
$$f(t) = f_g(t) + \underline{f}_\lambda(t),$$
$$f_\lambda(t) = N(H, t) \cdot \underline{\lambda}(t)$$
- Reward function:
 - Penalize $\|\dot{p}_{Rill}\| < \varepsilon$ (would be interesting to see if agent comes up with the same strategy)
 - Can also warm start it with Imitation learning: reward for mimicking the optimizer output
- Load tracking guaranteed by construction ($G \cdot N = 0$)
- Doesn't handle desynchronization
- A possible extension would be adding more terms to the reward function to account for fixed-wings dynamics (e.g. minimum turning radius)



Original Plan

Formulation 2: Residual RL

- Action Space: $\delta f \in \mathbb{R}^{3n}$
 - Keep wrench controller and optimizer. Run decentralized
$$f(t) = f_g(t) + f_\lambda(t) + \underline{\delta f(t)}$$
- Reward function:
 - Penalize load tracking error + Penalize $\|\delta f\|$
- Handles desynchronization
- Stage 1: Train formulation 2
- Stage 2: Remove optimizer and train formulation 1
- Could also work the other way around(would be better for simulation time) or combined in one phase



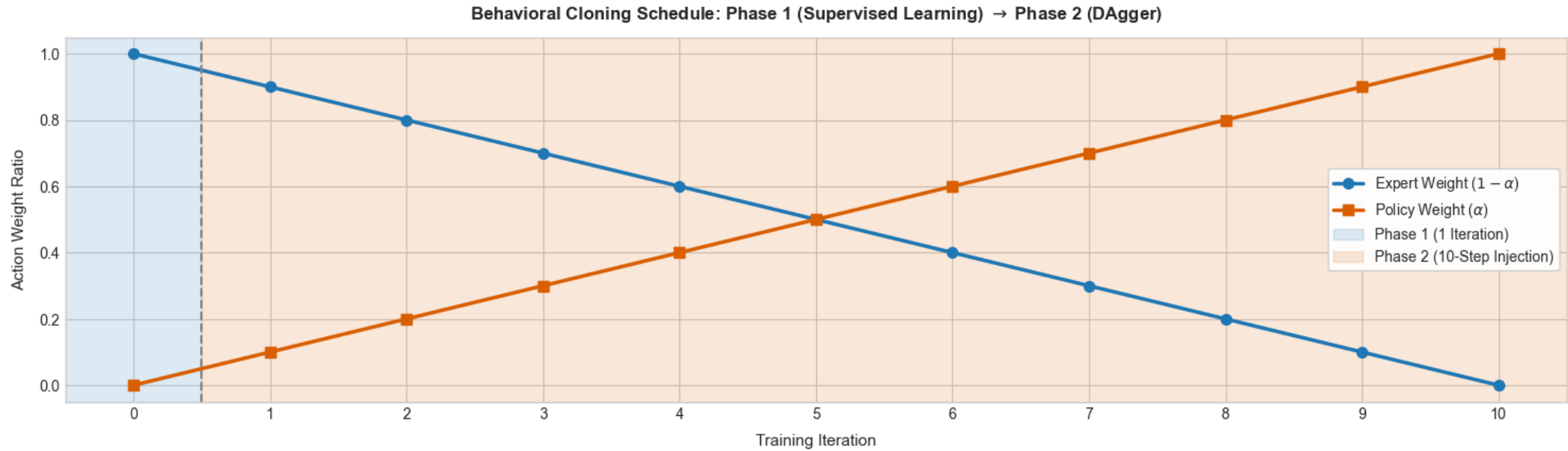
Actual

- Doing formulation 2 using the classical controller + optimizer(expert) takes very long
- Because it's decentralized the optimizer has to run 4 times per each step
- Episode length of 25 seconds takes 1-2 minutes in actual runtime



Formulation 1 status

- So I had to do formulation one first
 - Step 1 : Imitation learning/Behavioral cloning(BC): train a NN to mimic the optimizer
 - Phase 1 : Supervised learning : collect states and labels from expert and train
 - Phase 2 : DAgger: $\text{Action} = \beta * \text{expert action} + (1 - \beta) * \text{policy action}$
 - Slide β from 1 to 0 and train after each iteration
 - Action is the full lambda vector
 - Input to network is the full velocity vector estimated from the load state and wrench, previous lambda and “clock features/phases”
 - Full velocity vector is calculated analytically like the optimizer , I tested with multiple setups here



Mechanics: Phase 1 Supervised Learning vs. Phase 2 DAGger

Phase 1: Initial Pass (Iter 0)

- Action: $a_0 = a_0^*$ ($\alpha = 0$)
- Goal: Pure expert trajectory collection & initial fitting of π_{θ_0} .
- Result: Baseline offline policy.



Phase 2: Policy Action Injection (Iter 1–10)

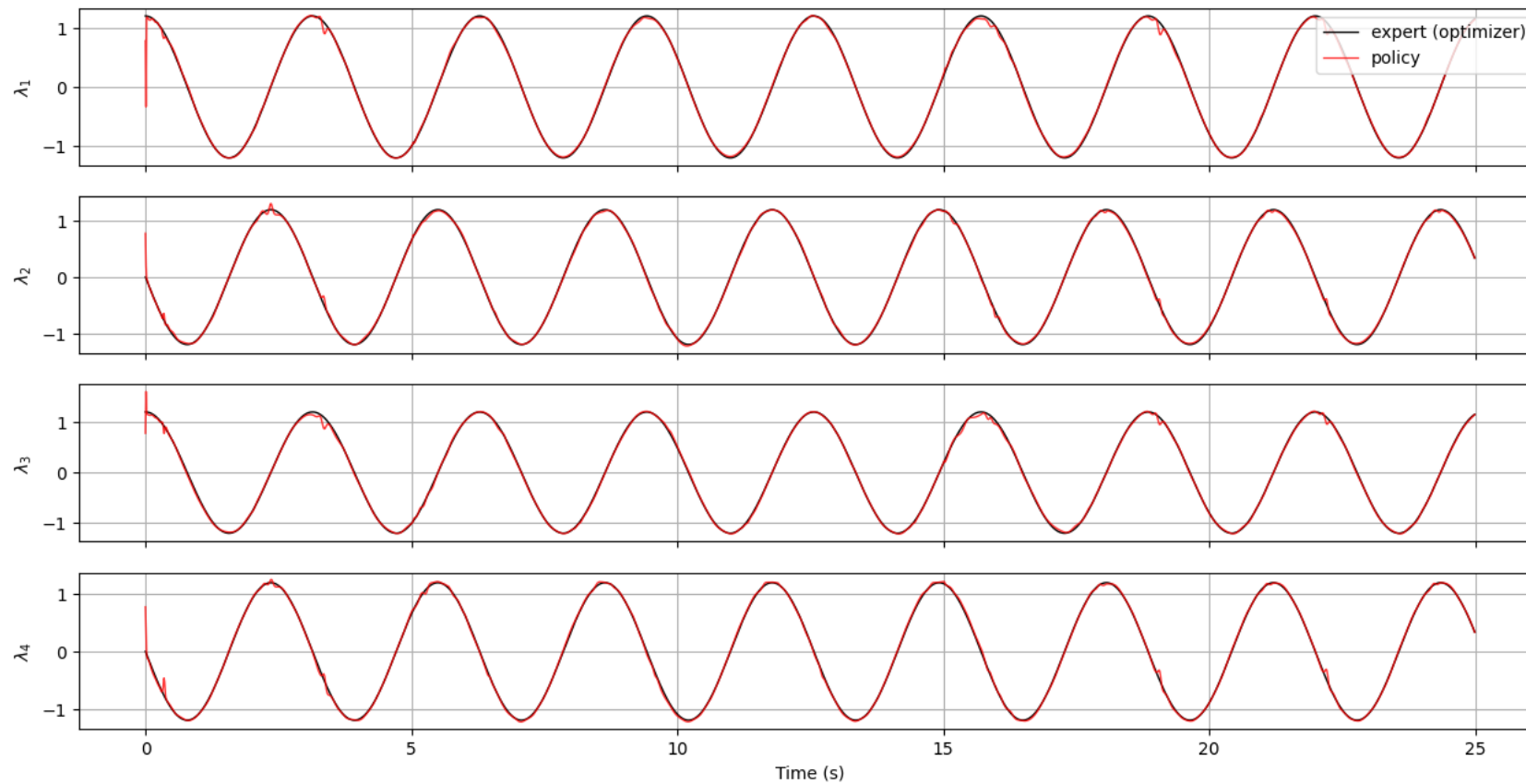
- Action: $a_t = (1 - \alpha)a_t^* + \alpha\pi(s_t)$
- Visited States: α increases 0.1 → 1.0.
- Goal: Exposes policy to its own mistakes while expert re-labels s_{t+1} with a_{t+1}^* .

Data is collected and added to the Dataset
The Network is retrained on the Aggregated Dataset.



Formulation 1 status

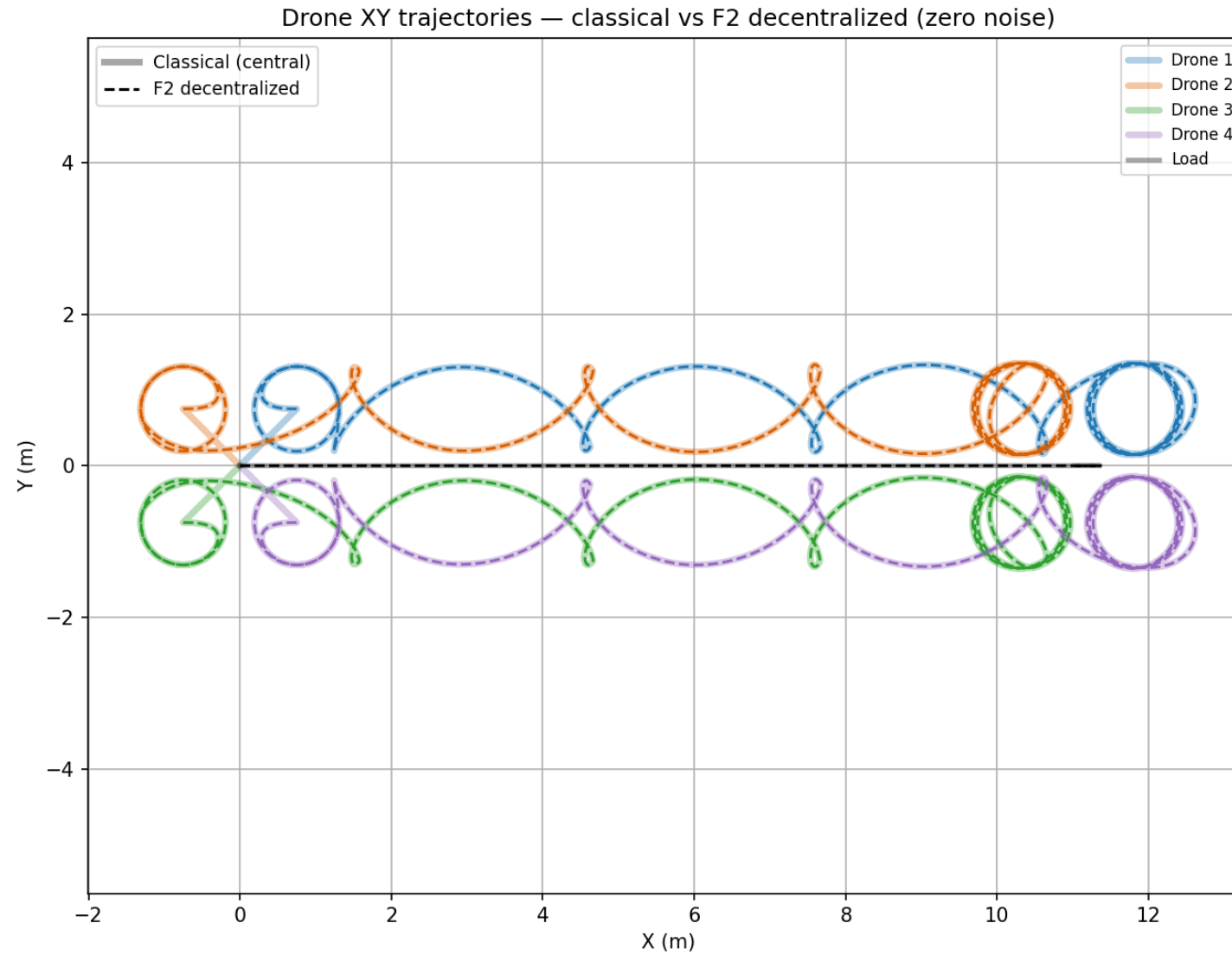
Expert vs policy lambda





Formulation 1 status

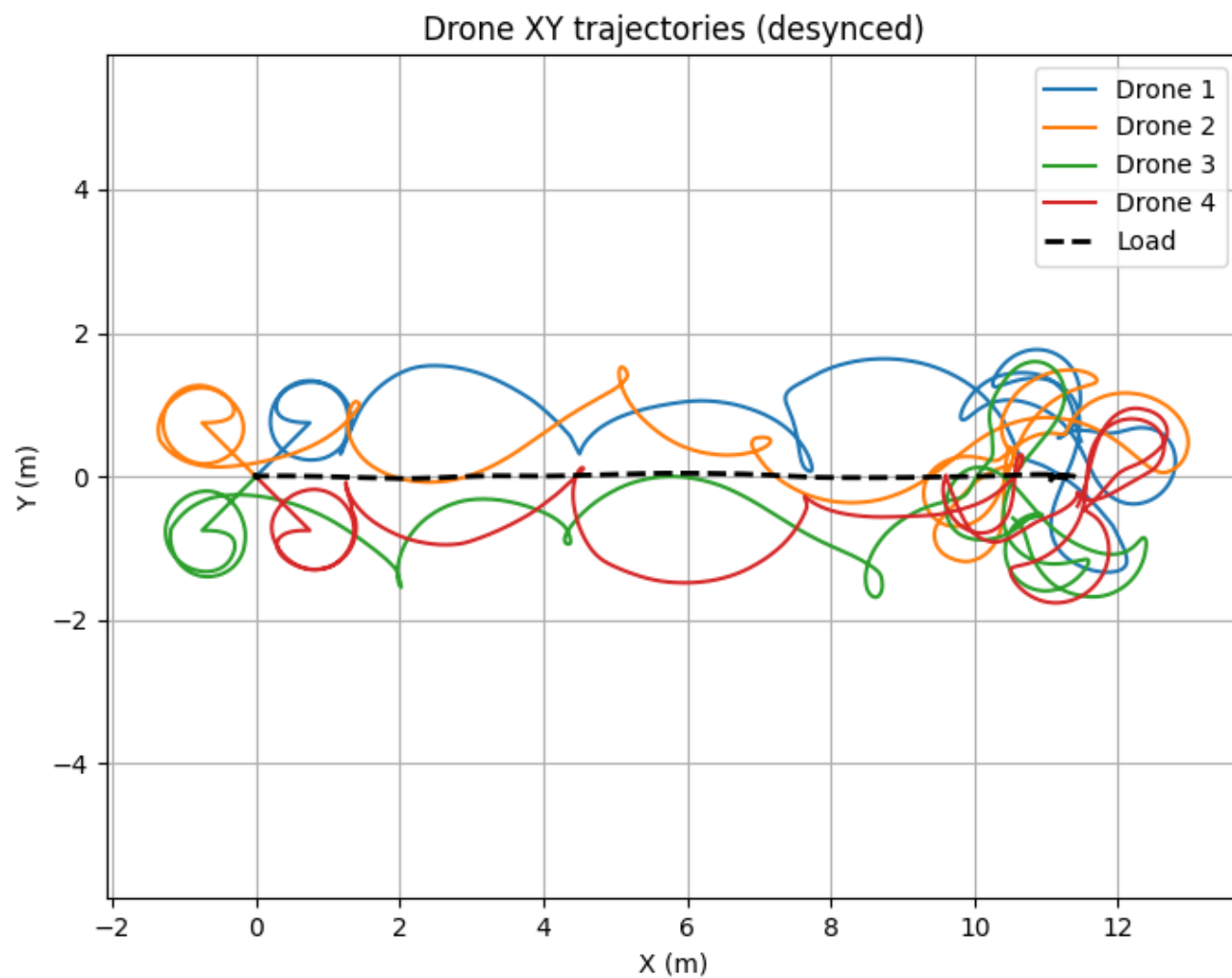
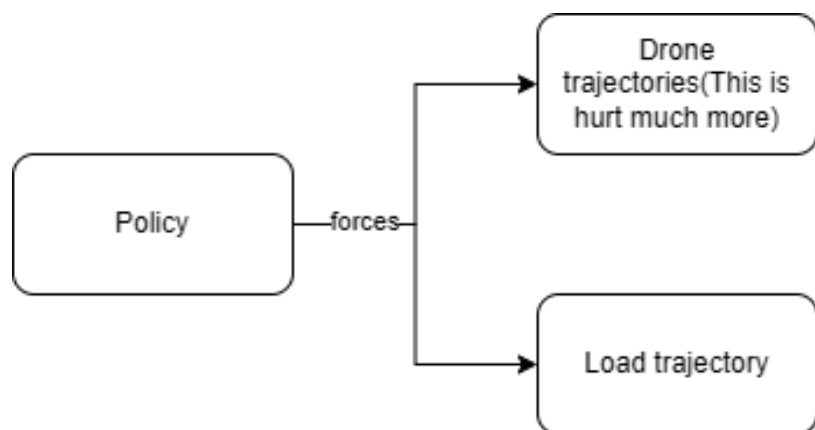
- Flying with the policy alone is successful
- So far BC with DAgger alone is producing acceptable results
- Episode length of 25 seconds takes ~ 7 seconds in actual runtime(10x faster)
- I trained on limited trajectories, so generalization still needs to be done
- As we generalize and add noise, RL becomes necessary
- RL objective regularized(TRPO relaxed-like) or just normal warm starting.
- Reward function design strategy:
 - Study/analyze the failure modes of BC alone and choose a reward function that fights it
 - It will have to do with jittering and non smooth movement in general*(similar to optimizer). $R = -w_{\text{lam}} * \sum_i (\text{lam}_{i_t} - \text{lam}_{i_prev})^2$

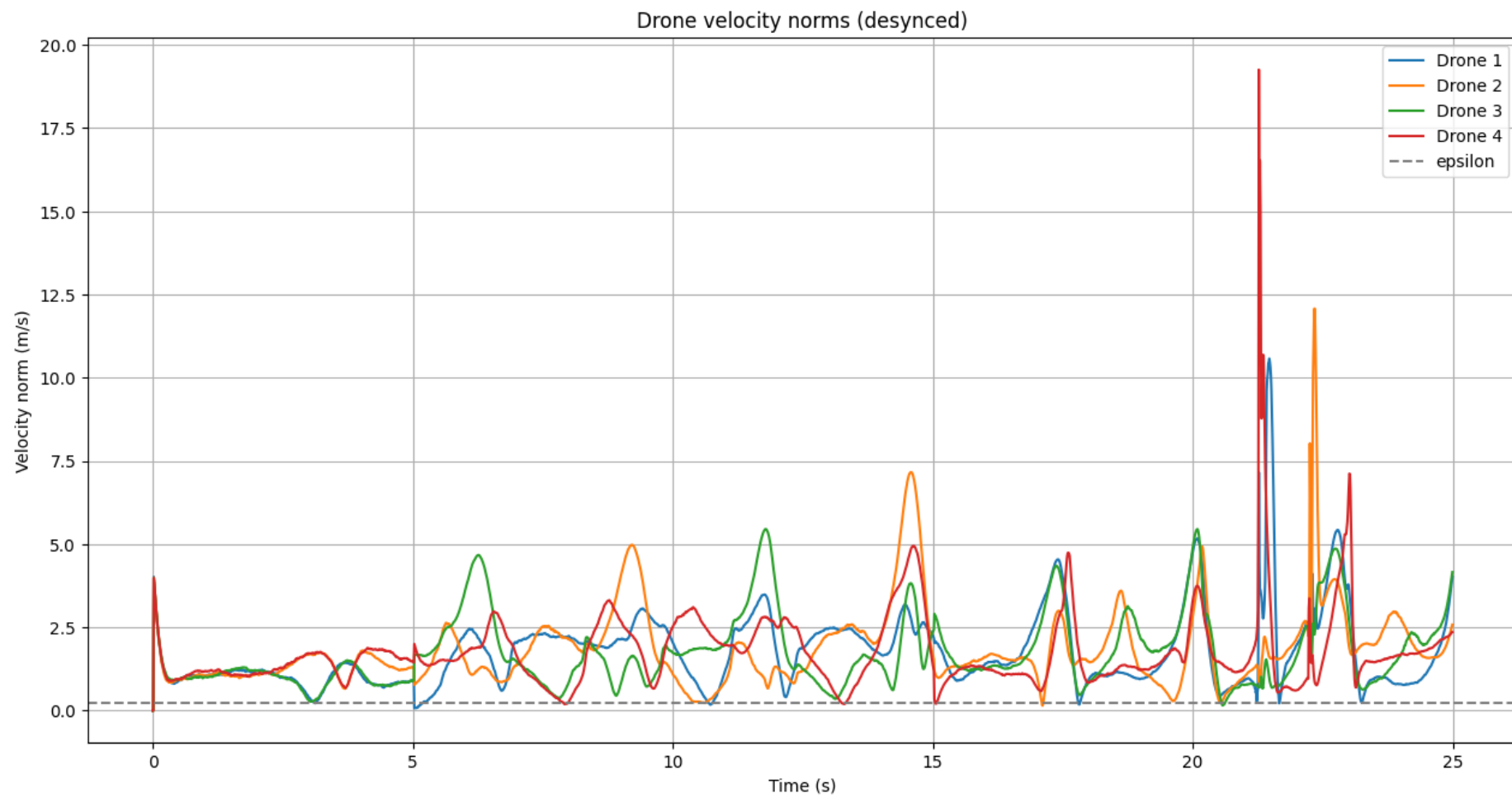


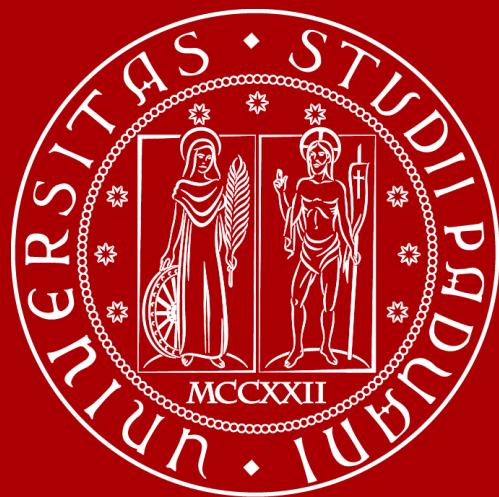


Formulation 2 status

- Currently I'm injecting noise in the load state and delay in some drones' clocks
- BC with DAgger alone completely failing(not sure if it's even helping)
- Still analyzing failure modes to try and design a good reward function
- Load tracking as a reward struggles due to the decoupling of drone trajectories and load trajectory
- A good reward would encourage the drones to be close to a mathematical description of the de-noised loitering







UNIVERSITÀ
DEGLI STUDI
DI PADOVA